

of files and subfolders; it's kept in a model object. The view object displays the data in whatever way the user chooses: as a list, as icons, as a detailed list, and so on. The controller tells the view to display the information in the layout chosen by the user.

Statistical Display Example

As a more extensive example of the MVC pattern, let's consider a program that simulates rolling a pair of dice many times and shows the results. In each roll we'll add the values of the two dice, so the sum—which we'll call an *outcome*—must be between 2 and 12. The data consists of the count of the number of times each outcome is rolled and the percentage of the total number of rolls each outcome makes up. The program can display this data in three different representations: a bar chart, a pie chart, and a text table. It defaults to a bar chart and displays the result after simulating rolling a pair of dice 2,500 times. Since this program is just intended as a working demonstration of the MVC pattern, we'll generate the output using `pygame` and `pygame_widgets`. For more professional-looking charts and displays, I suggest that you look into Python data visualization libraries such as `Matplotlib`, `Seaborn`, `Plotly`, `Bokeh`, and others that are designed for this purpose.

[Figure 17-3](#) shows the data displayed as a bar chart.